วิชาชีพชั้นสูงชั้นสูง พุทธศักราช 2567

ประเภทวิชา อุตสาหกรรมดิจิทัลและเทคโนโลยีสารสนเทศ กลุ่มอาชีพ ซอฟต์แวร์และการประยุกต์

สาขาวิชา เทคโนโลยีสารสนเทศ

รหัสวิชา 31901-2006 ชื่อวิชา การพัฒนาซอฟต์แวร์เชิงวัตถุ

ทฤษฎี 1 ชั่วโมง/สัปดาห์ ปฏิบัติ 4 ชั่วโมง/สัปดาห์ จำนวน 3 หน่วยกิต

อ้างอิงมาตรฐาน

-

ผลลัพธ์การเรียนรู้ระดับรายวิชา

พัฒนาซอฟต์แวร์ตามหลักการเชิงวัตถุ ด้วยความรับผิดชอบ ความละเอียด รอบคอบ และถูกต้อง

จุดประสงค์รายวิชา เพื่อให้

1. เข้าใจเกี่ยวกับหลักการพัฒนาซอฟต์แวร์เชิงวัตถุ
2. มีทักษะในการพัฒนาซอฟต์แวร์เชิงวัตถุ
3. มีเจตคติและกิจนิสัยที่ดีในการปฏิบัติงานด้วยความรับผิดชอบ ความละเอียด รอบคอบ และถูกต้อง
4. มีความสามารถประยุกต์ใช้ภาษาคอมพิวเตอร์เชิงวัตถุเพื่อพัฒนาซอฟต์แวร์

สมรรถนะรายวิชา

1. ประมวลความรู้เกี่ยวกับการพัฒนาซอฟต์แวร์ตามหลักการเชิงวัตถุ
2. พัฒนาซอฟต์แวร์ด้วยภาษาคอมพิวเตอร์เชิงวัตถุ
3. ประยุกต์ใช้ภาษาคอมพิวเตอร์เชิงวัตถุเพื่อพัฒนาซอฟต์แวร์

คำอธิบายรายวิชา

ศึกษาและปฏิบัติเกี่ยวกับหลักการพื้นฐานของการโปรแกรมเชิงวัตถุด้วยภาษาคอมพิวเตอร์เชิงวัตถุ การติดตั้งและกำหนดการทำงานของโปรแกรมที่เกี่ยวข้องกับการพัฒนาโปรแกรมเชิงวัตถุ ลำดับขั้นตอนการเขียนโปรแกรมเชิงวัตถุ การสร้างและเรียกใช้งาน Classes, Objects, Attributes, Methods, Constructors, Modifiers, Encapsulation, Packages, Inheritance, Polymorphism, Abstraction, Interface, Exceptions Handlings, Threads และเชื่อมต่อฐานข้อมูลในการพัฒนาซอฟต์แวร์เชิงวัตถุในงานอาชีพ

ตารางวิเคราะห์หน่วยการเรียนรู้

ผลลัพธ์การเรียนรู้ระดับรายวิชา พัฒนาซอฟต์แวร์ตามหลักการเชิงวัตถุ ด้วยความรับผิดชอบ ความละเอียด รอบคอบ และถูกต้อง

งานหลัก (หน่วยการเรียนรู้)	งานย่อย (รายการเรียนรู้)	ความรู้ (พุทธิพิสัย)	ทักษะ (ทักษะพิสัย)	ความสามารถในการประยุกต์ใช้	คุณลักษณะที่พึงประสงค์ (จิตพิสัย)	สมรรถนะย่อย (สมรรถนะประจำหน่วย)
หน่วยที่ 1: การประยุกต์ใช้หลักการและแนวคิดการเขียนโปรแกรมเชิงวัตถุ	1.1 หลักการพื้นฐานของการโปรแกรมเชิงวัตถุ 1.2 การติดตั้งและกำหนดการทำงานของเครื่องมือพัฒนาโปรแกรม 1.3 โครงสร้างและลำดับขั้นตอนการเขียนโปรแกรมเชิงวัตถุ	- อธิบาย หลักการพื้นฐาน 4 เสาหลักของ OOP (Encapsulation, Inheritance, Polymorphism, Abstraction) ได้ถูกต้องครบถ้วน - เปรียบเทียบความแตกต่างระหว่าง Class และ Object ได้ - สามารถ วิเคราะห์ความแตกต่างระหว่างการเขียนโปรแกรมเชิงโครงสร้าง (Procedural) และเชิงวัตถุ (OOP) ได้	- ปฏิบัติการติดตั้งและตั้งค่า (Setup) เครื่องมือพัฒนาโปรแกรมและสภาพแวดล้อม (Environment) ให้พร้อมใช้งานได้ตามขั้นตอนที่กำหนด - สามารถใช้เทคโนโลยี AI (Gemini) เขียนคำสั่ง (Prompt) เพื่อสืบค้นและสรุปสาระสำคัญของ OOP ได้ใจความที่ถูกต้องและน่าเชื่อถือ - สามารถใช้ GitLab สร้างกลุ่มงาน (Group) และวางแผนงานผ่าน	- สามารถประยุกต์ใช้แนวคิด OOP ในการออกแบบโครงสร้างโปรแกรมเบื้องต้นเพื่อแก้ปัญหาได้อย่างสมเหตุสมผล	- ผู้เรียน แสดงความรับผิดชอบ โดยการเข้าเรียนตรงเวลา เตรียมความพร้อมอุปกรณ์ และ ส่งงานที่ได้รับมอบหมายได้ครบถ้วนตามกำหนดเวลา - ผู้เรียน ทำงานร่วมกับ AI ปฏิบัติตามจริยธรรม โดยการตรวจสอบความถูกต้องของข้อมูล (Fact-check) และระบุแหล่งที่มา โดยไม่คัดลอกผลลัพธ์มาใช้งานโดยตรง	1. แสดงความรู้และวิเคราะห์หลักการพื้นฐานของการเขียนโปรแกรมเชิงวัตถุ (OOP) เปรียบเทียบกับเชิงโครงสร้าง และจำแนกองค์ประกอบของ Class และ Object ตามหลักการได้ถูกต้อง 2. ติดตั้งและตั้งค่าสภาพแวดล้อม (Environment Setup) สำหรับการพัฒนาซอฟต์แวร์เชิงวัตถุ และเขียนคำสั่งโปรแกรมเบื้องต้นเพื่อทดสอบระบบได้สำเร็จตามคู่มือปฏิบัติงาน

		- สามารถ จำแนกองค์ประกอบของ Class และ Object ได้ถูกต้อง	ระบบ Issue Board ให้สอดคล้องกับภารกิจที่ได้รับมอบหมาย - เขียนและประมวลผล (Run) โปรแกรมเชิงวัตถุเบื้องต้น (เช่น Hello World) ได้ถูกต้องโดยปราศจากข้อผิดพลาด (Syntax Error)		- ผู้เรียน แสดงทัศนคติที่ดี โดยการรับฟังความคิดเห็นของสมาชิกและช่วยเหลือเพื่อนร่วมทีมอย่างสร้างสรรค์	3. วางแผนและบริหารจัดการลำดับขั้นตอนการพัฒนาซอฟต์แวร์ โดยประยุกต์ใช้ระบบควบคุมเวอร์ชัน (GitLab) และเครื่องมือ AI ช่วยสนับสนุนการทำงานเป็นทีมได้อย่างมีประสิทธิภาพ
หน่วยที่ 2: การสร้างและใช้งานคลาสและอ็อบเจกต์	2.1 คลาส (Classes) 2.2 อ็อบเจกต์ (Objects) 2.3 แอตทริบิวต์ (Attributes) 2.4 เมธอด (Methods) 2.5 คอนสตรักเตอร์ (Constructors)	- ความสัมพันธ์ระหว่างคลาสและอ็อบเจกต์ - องค์ประกอบของคลาส (Attribute, Method) - หน้าที่ของคอนสตรักเตอร์	- การสร้างคลาส - การกำหนดแอตทริบิวต์และเมธอด - การสร้างอ็อบเจกต์และเรียกใช้งาน - การเขียนคอนสตรักเตอร์	- การจำลองวัตถุในสถานการณ์จริงด้วยคลาสและอ็อบเจกต์	- ความละเอียดในการออกแบบโครงสร้างคลาส - การตระหนักถึงความถูกต้องในการใช้งานอ็อบเจกต์	1. ออกแบบและสร้างคลาสเพื่อจำลองคุณลักษณะและการทำงานของวัตถุ 2. สร้างและเรียกใช้งานอ็อบเจกต์ แอตทริบิวต์ เมธอด และคอนสตรักเตอร์ตามหลักการโปรแกรมเชิงวัตถุ
หน่วยที่ 3: การจัดการโครงสร้าง	3.1 ตัวกำหนดการเข้าถึง (Modifiers)	- ประเภทของตัวกำหนดการเข้าถึง	- การกำหนดระดับการเข้าถึงแอตทริบิวต์และเมธอด	- การประยุกต์ใช้หลักการห่อหุ้มในการ	- การเห็นความสำคัญของการปกป้องข้อมูล	1. แสดงความรู้เกี่ยวกับการควบคุมการเข้าถึงและการห่อหุ้มข้อมูล

โปรแกรมเชิงวัตถุ	3.2 การห่อหุ้ม (Encapsulation) 3.3 แพ็กเกจ (Packages)	- ประโยชน์ของหลักการห่อหุ้ม - หลักการจัดระเบียบโค้ดด้วยแพ็กเกจ	- การเขียนโค้ดตามหลักการห่อหุ้ม - การสร้างและนำเข้าแพ็กเกจ	ออกแบบคลาสที่มีความปลอดภัย	- มีวินัยในการจัดโครงสร้างโปรแกรม	2. จัดการโครงสร้างโปรแกรมด้วยแพ็กเกจตามหลักปฏิบัติที่ดี
หน่วยที่ 4: การประยุกต์ใช้คุณสมบัติการสืบทอดและการพ้องรูป	4.1 การสืบทอด (Inheritance) 4.2 การพ้องรูป (Polymorphism)	- แนวคิดและความสัมพันธ์แบบแม่-ลูก (Parent-Child) - หลักการทำงานของการพ้องรูป	- การสร้างคลาสลูกที่สืบทอดคุณสมบัติ - การเขียนโค้ดเรียกใช้เมธอดที่มีลักษณะพ้องรูป - การ Override เมธอด	- การประยุกต์ใช้การสืบทอดเพื่อสร้างลำดับชั้นของคลาส - การแก้ปัญหาโดยใช้การพ้องรูปเพื่อลดความซับซ้อนของโค้ด	- การตระหนักถึงประโยชน์ของการนำโค้ดกลับมาใช้ใหม่ (Code Reusability) - ความมุ่งมั่นในการเขียนโค้ดที่ยืดหยุ่น	1. ประยุกต์ใช้คุณสมบัติการสืบทอด (Inheritance) เพื่อสร้างคลาสใหม่บนพื้นฐานของคลาสที่มีอยู่ 2. พัฒนาโปรแกรมที่มีความยืดหยุ่นโดยใช้หลักการพ้องรูป (Polymorphism)
หน่วยที่ 5: การออกแบบด้วยแนวคิดนามธรรมและอินเทอร์เฟซ	5.1 แนวคิดนามธรรม (Abstraction) 5.2 อินเทอร์เฟซ (Interface)	- ความสำคัญของแนวคิดนามธรรม - ความแตกต่างระหว่าง Abstract Class และ Interface	- การสร้าง Abstract Class และ Abstract Method - การสร้างและนำ Interface ไปใช้งาน (Implement)	- การออกแบบโครงสร้างโปรแกรมโดยใช้ Abstract Class และ Interface	- การเห็นประโยชน์ของการออกแบบโดยมุ่งเน้นที่ "อะไร" (What) มากกว่า "อย่างไร" (How) - ความรับผิดชอบในการ Implement เมธอดตาม Interface	1. แสดงความรู้ความเข้าใจในการออกแบบซอฟต์แวร์ด้วยแนวคิดนามธรรม (Abstraction) 2. กำหนดและใช้งานอินเทอร์เฟซ (Interface) เพื่อสร้างข้อกำหนดการทำงานของคลาส

หน่วยที่ 6: การจัดการข้อผิดพลาดและโปรแกรมแบบมัลติเทรด	6.1 การจัดการข้อผิดพลาด (Exceptions Handlings) 6.2 การทำงานแบบหลายเทรด (Threads)	- ความจำเป็นของการจัดการข้อผิดพลาด - แนวคิดพื้นฐานของการทำงานแบบหลายเทรด	- การเขียนโค้ดดักจับและจัดการข้อผิดพลาด (try-catch) - การสร้างและสั่งรันเทรดอย่างง่าย	- การประยุกต์ใช้การจัดการข้อผิดพลาดเพื่อสร้างโปรแกรมที่มีเสถียรภาพ	- ความรอบคอบในการคาดการณ์ข้อผิดพลาด - การตระหนักถึงความซับซ้อนของการทำงานแบบหลายเทรด	1. พัฒนาโปรแกรมที่มีความทนทานต่อข้อผิดพลาดโดยใช้กลไก Exceptions Handling 2. สร้างและจัดการเทรด (Threads) สำหรับการทำงานแบบขนานเบื้องต้น
หน่วยที่ 7: การเชื่อมต่อฐานข้อมูลในโปรแกรมเชิงวัตถุ	7.1 หลักการเชื่อมต่อฐานข้อมูล 7.2 การเรียกใช้คำสั่ง SQL ผ่านโปรแกรม 7.3 การจัดการผลลัพธ์จากฐานข้อมูล	- ขั้นตอนการเชื่อมต่อฐานข้อมูล - ประเภทของคำสั่ง SQL ในการจัดการข้อมูล	- การเขียนโค้ดสร้างการเชื่อมต่อฐานข้อมูล - การเขียนโค้ดส่งคำสั่ง SQL - การเขียนโค้ดรับและแสดงผลข้อมูล	- การประยุกต์ใช้ความรู้เพื่อสร้างโปรแกรมจัดการข้อมูลพื้นฐาน (เพิ่ม, ลบ, แก้ไข, ค้นหา) ในฐานข้อมูล	- การเห็นความสำคัญของการจัดการทรัพยากรการเชื่อมต่อ - ความรับผิดชอบต่อความปลอดภัยของข้อมูล	1. พัฒนาโปรแกรมเชิงวัตถุเพื่อเชื่อมต่อกับระบบฐานข้อมูล 2. ประยุกต์ใช้โปรแกรมเชิงวัตถุในการจัดการข้อมูลผ่านคำสั่ง SQL ในงานอาชีพ
งานหลัก (หน่วยการเรียนรู้)	**งานย่อย (รายการเรียนรู้)**	**ความรู้ (พุทธิพิสัย)**	**ทักษะ (ทักษะพิสัย)**	**ความสามารถในการประยุกต์ใช้**	**คุณลักษณะที่พึงประสงค์ (จิตพิสัย)**	**สมรรถนะย่อย (สมรรถนะประจำหน่วย)**
หน่วยที่ 1 การประยุกต์ใช้หลักการและแนวคิดการ	1.1 หลักการพื้นฐานของการโปรแกรมเชิงวัตถุ 1.2 การติดตั้งและกำหนดการทำงาน	- หลักการและแนวคิดพื้นฐานของ OOP - ขั้นตอนการติดตั้งและตั้งค่าเครื่องมือ	- การติดตั้งเครื่องมือ - การกำหนดค่าเบื้องต้น - การเขียนและรันโปรแกรมอย่างง่าย	- การใช้เครื่องมือในการสร้างและรันโปรแกรมอย่างง่าย	- การเห็นความสำคัญของหลักการโปรแกรมเชิงวัตถุ	1. แสดงความรู้เกี่ยวกับหลักการพื้นฐานของการโปรแกรมเชิงวัตถุ

เขียนโปรแกรมเชิงวัตถุ	ของเครื่องมือพัฒนาโปรแกรม 1.3 โครงสร้างและลำดับขั้นตอนการเขียนโปรแกรมเชิงวัตถุ	- โครงสร้างพื้นฐานของโปรแกรมเชิงวัตถุ			- ความรอบคอบในการติดตั้งและตั้งค่าเครื่องมือ	2. ติดตั้งและเตรียมความพร้อมสภาพแวดล้อมสำหรับการพัฒนาโปรแกรมเชิงวัตถุ
หน่วยที่ 2 การสร้างและใช้งานคลาสและอ็อบเจกต์	2.1 คลาส (Classes) 2.2 อ็อบเจกต์ (Objects) 2.3 แอตทริบิวต์ (Attributes) 2.4 เมธอด (Methods) 2.5 คอนสตรักเตอร์ (Constructors)	- ความสัมพันธ์ระหว่างคลาสและอ็อบเจกต์ - องค์ประกอบของคลาส (Attribute, Method) - หน้าที่ของคอนสตรักเตอร์	- การสร้างคลาส - การกำหนดแอตทริบิวต์และเมธอด - การสร้างอ็อบเจกต์และเรียกใช้งาน - การเขียนคอนสตรักเตอร์	- การจำลองวัตถุในสถานการณ์จริงด้วยคลาสและอ็อบเจกต์	- ความละเอียดในการออกแบบโครงสร้างคลาส - การตระหนักถึงความถูกต้องในการใช้งานอ็อบเจกต์	1. ออกแบบและสร้างคลาสเพื่อจำลองคุณลักษณะและการทำงานของวัตถุ 2. สร้างและเรียกใช้งานอ็อบเจกต์ แอตทริบิวต์ เมธอด และคอนสตรักเตอร์ตามหลักการโปรแกรมเชิงวัตถุ
หน่วยที่ 3 การจัดการโครงสร้างโปรแกรมเชิงวัตถุ	3.1 ตัวกำหนดการเข้าถึง (Modifiers) 3.2 การห่อหุ้ม (Encapsulation) 3.3 แพ็กเกจ (Packages)	- ประเภทของตัวกำหนดการเข้าถึง - ประโยชน์ของหลักการห่อหุ้ม - หลักการจัดระเบียบโค้ดด้วยแพ็กเกจ	- การกำหนดระดับการเข้าถึงแอตทริบิวต์และเมธอด - การเขียนโค้ดตามหลักการห่อหุ้ม	- การประยุกต์ใช้หลักการห่อหุ้มในการออกแบบคลาสที่มีความปลอดภัย	- การเห็นความสำคัญของการปกป้องข้อมูล - มีวินัยในการจัดโครงสร้างโปรแกรม	1. แสดงความรู้เกี่ยวกับการควบคุมการเข้าถึงและการห่อหุ้มข้อมูล 2. จัดการโครงสร้างโปรแกรมด้วยแพ็กเกจตามหลักปฏิบัติที่ดี

			- การสร้างและนำเข้าแพ็กเกจ			
หน่วยที่ 4 การประยุกต์ใช้คุณสมบัติการสืบทอดและการพ้องรูป	4.1 การสืบทอด (Inheritance) 4.2 การพ้องรูป (Polymorphism)	- แนวคิดและความสัมพันธ์แบบแม่-ลูก (Parent-Child) - หลักการทำงานของการพ้องรูป	- การสร้างคลาสลูกที่สืบทอดคุณสมบัติ - การเขียนโค้ดเรียกใช้เมธอดที่มีลักษณะพ้องรูป - การ Override เมธอด	- การประยุกต์ใช้การสืบทอดเพื่อสร้างลำดับชั้นของคลาส - การแก้ปัญหาโดยใช้การพ้องรูปเพื่อลดความซับซ้อนของโค้ด	- การตระหนักถึงประโยชน์ของการนำโค้ดกลับมาใช้ใหม่ (Code Reusability) - ความมุ่งมั่นในการเขียนโค้ดที่ยืดหยุ่น	1. ประยุกต์ใช้คุณสมบัติการสืบทอด (Inheritance) เพื่อสร้างคลาสใหม่บนพื้นฐานของคลาสที่มีอยู่ 2. พัฒนาโปรแกรมที่มีความยืดหยุ่นโดยใช้หลักการพ้องรูป (Polymorphism)
หน่วยที่ 5 การออกแบบด้วยแนวคิดนามธรรมและอินเทอร์เฟซ	5.1 แนวคิดนามธรรม (Abstraction) 5.2 อินเทอร์เฟซ (Interface)	- ความสำคัญของแนวคิดนามธรรม - ความแตกต่างระหว่าง Abstract Class และ Interface	- การสร้าง Abstract Class และ Abstract Method - การสร้างและนำ Interface ไปใช้งาน (Implement)	- การออกแบบโครงสร้างโปรแกรมโดยใช้ Abstract Class และ Interface	- การเห็นประโยชน์ของการออกแบบโดยมุ่งเน้นที่ "อะไร" (What) มากกว่า "อย่างไร" (How) - ความรับผิดชอบในการ Implement เมธอดตาม Interface	1. แสดงความรู้ความเข้าใจในการออกแบบซอฟต์แวร์ด้วยแนวคิดนามธรรม (Abstraction) 2. กำหนดและใช้งานอินเทอร์เฟซ (Interface) เพื่อสร้างข้อกำหนดการทำงานของคลาส
หน่วยที่ 6 การจัดการข้อผิดพลาดและ	6.1 การจัดการข้อผิดพลาด (Exceptions Handlings)	- ความจำเป็นของการจัดการข้อผิดพลาด - แนวคิดพื้นฐานของการทำงานแบบหลายเทรด	- การเขียนโค้ดดักจับและจัดการข้อผิดพลาด (try-catch)	- การประยุกต์ใช้การจัดการข้อผิดพลาดเพื่อสร้างโปรแกรมที่มีเสถียรภาพ	- ความรอบคอบในการคาดการณ์ข้อผิดพลาด	1. พัฒนาโปรแกรมที่มีความทนทานต่อข้อผิดพลาดโดยใช้กลไก Exceptions Handling

โปรแกรมแบบมัลติเทรด	6.2 การทำงานแบบหลายเทรด (Threads)		- การสร้างและสั่งรันเทรดอย่างง่าย		- การตระหนักถึงความซับซ้อนของการทำงานแบบหลายเทรด	2. สร้างและจัดการเทรด (Threads) สำหรับการทำงานแบบขนานเบื้องต้น
หน่วยที่ 7 การเชื่อมต่อฐานข้อมูลในโปรแกรมเชิงวัตถุ	7.1 หลักการเชื่อมต่อฐานข้อมูล 7.2 การเรียกใช้คำสั่ง SQL ผ่านโปรแกรม 7.3 การจัดการผลลัพธ์จากฐานข้อมูล	- ขั้นตอนการเชื่อมต่อฐานข้อมูล - ประเภทของคำสั่ง SQL ในการจัดการข้อมูล	- การเขียนโค้ดสร้างการเชื่อมต่อฐานข้อมูล - การเขียนโค้ดส่งคำสั่ง SQL - การเขียนโค้ดรับและแสดงผลข้อมูล	- การประยุกต์ใช้ความรู้เพื่อสร้างโปรแกรมจัดการข้อมูลพื้นฐาน (เพิ่ม, ลบ, แก้ไข, ค้นหา) ในฐานข้อมูล	- การเห็นความสำคัญของการจัดการทรัพยากรการเชื่อมต่อ - ความรับผิดชอบต่อความปลอดภัยของข้อมูล	1. พัฒนาโปรแกรมเชิงวัตถุเพื่อเชื่อมต่อกับระบบฐานข้อมูล 2. ประยุกต์ใช้โปรแกรมเชิงวัตถุในการจัดการข้อมูลผ่านคำสั่ง SQL ในงานอาชีพ

ตารางวิเคราะห์พฤติกรรมการเรียนรู้

รหัสวิชา 31901-2006 **ชื่อวิชา** การพัฒนาซอฟต์แวร์เชิงวัตถุ

ทฤษฎี 1 ชั่วโมง/สัปดาห์ **ปฏิบัติ** 4 ชั่วโมง/สัปดาห์ **จำนวน** 3 หน่วยกิต

หน่วยการเรียนรู้	พุทธิพิสัย						ทักษะพิสัย	จิตพิสัย	ประยุกต์ใช้	รวม	จำนวนชั่วโมง (ท/ป)
	ความรู้	ความเข้าใจ	การนำไปใช้	การวิเคราะห์	การประเมินค่า	การสร้างสรรค์					
1. การประยุกต์ใช้หลักการและแนวคิดฯ	-	6	1	2	-	-	4	3	1	**12**	1.0 / 4.0
2. การสร้างและจัดการคลาสและอ็อบเจกต์	-	2	2	-	-	-	1	1	1	**7**	2.3 / 7.7
3. การจัดการการเข้าถึงข้อมูลฯ	-	2	2	-	-	-	1	1	1	**7**	2.0 / 8.0
4. การประยุกต์ใช้คุณสมบัติ Inheritance และ Polymorphism	-	1	2	1	-	-	1	1	1	**7**	2.0 / 8.0
5. การออกแบบโปรแกรมด้วย Abstraction และ Interface	-	-	2	1	-	-	1	1	1	**6**	2.0 / 8.0
6. การจัดการข้อผิดพลาดและโปรแกรมแบบมัลติเธรด	-	2	2	-	-	-	1	1	1	**7**	2.0 / 8.0
7. การเชื่อมต่อและจัดการฐานข้อมูล	-	1	2	-	-	-	1	1	1	**6**	4.0 / 16.0
รวม	**1**	**9**	**12**	**2**	**-**	**-**	**7**	**7**	**7**	**45**	
รวมทั้งรายวิชา											**15.3 / 59.7**

ประเมินผลลัพธ์การเรียนรู้ระดับรายวิชา เมื่อเรียนรายวิชานี้สำเร็จแล้ว ผู้เรียนสามารถ **พัฒนาซอฟต์แวร์ตามหลักการเชิงวัตถุ** ที่สามารถเชื่อมต่อฐานข้อมูลได้ ด้วยความรับผิดชอบและถูกต้องตามหลักการ

ตารางเวลาเรียนหน่วยการเรียนรู้

รหัสวิชา 31901-2006 **ชื่อวิชา** การพัฒนาซอฟต์แวร์เชิงวัตถุ (ทฤษฎี 1 ชม./สัปดาห์, ปฏิบัติ 4 ชม./สัปดาห์)

หน่วยที่	หน่วยการเรียนรู้ (งานย่อย/หัวข้อเรื่อง)	สัปดาห์ที่	เวลาเรียน (ชม.)		
			ทฤษฎี	ปฏิบัติ	รวม
1	**การประยุกต์ใช้หลักการและแนวคิดการเขียนโปรแกรมเชิงวัตถุ**	**1**			
	1.1 หลักการพื้นฐานของการโปรแกรมเชิงวัตถุ (OOP)		0.5	1.0	1.5
	1.2 การติดตั้งและกำหนดค่าสภาพแวดล้อมการพัฒนา (IDE)		0.5	2.0	2.5
	1.3 ลำดับขั้นตอนการพัฒนาโปรแกรมเชิงวัตถุ		-	1.0	1.0
2	**การสร้างและจัดการคลาสและอ็อบเจกต์**	**2 - 3**			
	2.1 คลาส (Classes) และ อ็อบเจกต์ (Objects)	2	0.5	2.0	2.5
	2.2 แอตทริบิวต์ (Attributes)		0.5	2.0	2.5
	2.3 เมธอด (Methods)	**3**	**1.0**	**3.0**	**4.0**
	2.4 คอนสตรัคเตอร์ (Constructors)		**0.3**	**0.7**	**1.0**
3	**การจัดการการเข้าถึงข้อมูลด้วย Encapsulation และ Package**	**4 - 5**			
	3.1 ตัวกำหนดการเข้าถึง (Access Modifiers)	4	0.5	1.5	2.0
	3.2 หลักการห่อหุ้ม (Encapsulation)		0.5	2.5	3.0
	3.3 แพ็กเกจ (Packages)	5	1.0	4.0	5.0
4	**การประยุกต์ใช้คุณสมบัติการสืบทอด (Inheritance) และพ้องรูป (Polymorphism)**	**6 - 7**			
	4.1 การสืบทอดคุณสมบัติ (Inheritance)	6	1.0	4.0	5.0
	4.2 การพ้องรูป (Polymorphism)	7	1.0	4.0	5.0
5	**การออกแบบโปรแกรมด้วย Abstraction และ Interface**	**8 - 9**			
	5.1 คลาสนามธรรม (Abstraction) และเมธอดนามธรรม	8	1.0	4.0	5.0
	5.2 อินเทอร์เฟซ (Interface)	9	1.0	4.0	5.0
6	**การจัดการข้อผิดพลาดและโปรแกรมแบบมัลติเธรด**	**10 - 11**			
	6.1 การจัดการข้อผิดพลาด (Exceptions Handlings)	10	1.0	4.0	5.0

	6.2 การทำงานแบบหลายสายงาน (Threads)	11	1.0	4.0	5.0
7	**การเชื่อมต่อและจัดการฐานข้อมูล**	**12 - 15**			
	7.1 หลักการเชื่อมต่อฐานข้อมูล (Database Connectivity)	12	1.0	4.0	5.0
	7.2 การจัดการข้อมูล (เพิ่ม ลบ แก้ไข ค้นหา)	13 - 15	3.0	12.0	15.0
	ประเมินผลปลายภาค	**16**	-	-	-
	รวมทั้งสิ้น		**15.3**	**59.7**	**75.0**

การวัดและประเมินผล

การวัดและประเมินผลการเรียนรู้ประจำหน่วยโดยใช้ชุดการสอนวิชาการพัฒนาซอฟต์แวร์เชิงวัตถุ รหัสวิชา 31901–2006 โดยใช้วิธีการสอนแบบ การเรียนรู้โดยใช้ปัญหาเป็นฐานร่วมกับผู้ช่วยอัจฉริยะ (PBL-AI Integrated Learning Model) มีการดำเนินการเก็บรวบรวมข้อมูลเพื่อการประเมินผลตามสภาพจริง (Authentic Assessment) ดังนี้

คะแนนเต็ม 100 คะแนน แบ่งเป็น

1. คะแนนด้านสมรรถนะและกระบวนการจัดการเรียนรู้ (K, S, Ap) ร้อยละ 60

เป็นการประเมินผลระหว่างการจัดการเรียนการสอนจากการปฏิบัติงานย่อยในแต่ละหน่วยการเรียนรู้ โดยพิจารณาจากผลงาน (Product) และกระบวนการ (Process) เช่น การเขียนโค้ด, การใช้เครื่องมือ IDE/GitLab, และการแก้ปัญหา

เครื่องมือที่ใช้ แบบทดสอบประจำหน่วย, ใบงาน, ใบมอบหมายงาน, และแบบประเมินสมรรถนะ (Rubrics)

เกณฑ์การคำนวณ

$$\text{คะแนนสมรรถนะ (60)} = \sum(\text{คะแนนเก็บระหว่างภาคของทุกหน่วยการเรียนรู้ K+S+Ap})$$

2. คะแนนด้านคุณธรรม จริยธรรม และกิจนิสัย (A) ร้อยละ 20

เป็นการประเมินพฤติกรรมการเรียนรู้และการทำงานร่วมกัน โดยเน้นความรับผิดชอบ ความซื่อสัตย์ และจริยธรรมในการใช้เทคโนโลยี AI (AI Ethics)

เครื่องมือที่ใช้ แบบสังเกตพฤติกรรม, แบบบันทึกการใช้งาน AI (AI Usage Log), และการตรวจสอบการเข้าเรียน

เกณฑ์การคำนวณ

$$\text{คะแนนจิตพิสัย (20)} = \sum(\text{คะแนนคุณธรรมจริยธรรมของทุกหน่วยการเรียนรู้ A})$$

3. การวัดผลสัมฤทธิ์ปลายภาคเรียน (Summative Assessment) ร้อยละ 20

เป็นการประเมินความรู้รวบยอดและการประยุกต์ใช้ความรู้ในการทำโครงงานหรือข้อสอบปลายภาค

เครื่องมือที่ใช้ แบบทดสอบวัดผลสัมฤทธิ์ปลายภาค หรือ โครงงานพัฒนาซอฟต์แวร์ (Mini Project)

ตารางคะแนนประจำหน่วย ผลลัพธ์การประเมินผลการเรียนรู้ตามสภาพจริง

งานย่อย (หัวข้อเรื่อง)	คะแนนเก็บระหว่างภาค (K+S+Ap)	คะแนนคุณธรรมจริยธรรม (A)	รวม
หน่วยที่ 1 การประยุกต์ใช้หลักการและแนวคิดการเขียนโปรแกรมเชิงวัตถุ			6
1.1 หลักการพื้นฐานของการโปรแกรมเชิงวัตถุ (OOP)	1	1	2
1.2 การติดตั้งและกำหนดค่าสภาพแวดล้อมการพัฒนา (IDE)	2	1	3
1.3 ลำดับขั้นตอนการพัฒนาโปรแกรมเชิงวัตถุ	0.5	0.5	1
หน่วยที่ 2 การสร้างและจัดการคลาสและอ็อบเจกต์			11
2.1 คลาส (Classes) และ อ็อบเจกต์ (Objects)	2	1	3
2.2 แอตทริบิวต์ (Attributes)	2	1	3
2.3 เมธอด (Methods)	3	1	4
2.4 คอนสตรัคเตอร์ (Constructors)	0.5	0.5	1
หน่วยที่ 3 การจัดการการเข้าถึงข้อมูลด้วย Encapsulation และ Package			11
3.1 ตัวกำหนดการเข้าถึง (Access Modifiers)	2	1	3
3.2 หลักการห่อหุ้ม (Encapsulation)	2	1	3
3.3 แพ็กเกจ (Packages)	4	1	5
หน่วยที่ 4 การประยุกต์ใช้คุณสมบัติการสืบทอด (Inheritance) และพ้องรูป (Polymorphism)			12
4.1 การสืบทอดคุณสมบัติ (Inheritance)	4	2	6
4.2 การพ้องรูป (Polymorphism)	4	2	6
หน่วยที่ 5 การออกแบบโปรแกรมด้วย Abstraction และ Interface			10
5.1 คลาสนามธรรม (Abstraction) และเมธอดนามธรรม	4	1	5
5.2 อินเทอร์เฟซ (Interface)	4	1	5
หน่วยที่ 6 การจัดการข้อผิดพลาดและโปรแกรมแบบมัลติเธรด			10
6.1 การจัดการข้อผิดพลาด (Exceptions Handlings)	4	1	5
6.2 การทำงานแบบหลายสายงาน (Threads)	4	1	5

หน่วยที่ 7 การเชื่อมต่อและจัดการฐานข้อมูล			20
7.1 หลักการเชื่อมต่อฐานข้อมูล (Database Connectivity)	4	2	6
7.2 การจัดการข้อมูล (เพิ่ม ลบ แก้ไข ค้นหา)	12	2	14
รวมคะแนนเก็บระหว่างภาค (ส่วนที่ 1 + ส่วนที่ 2)	**60**	**20**	**80**
สอบปลายภาค			**20**
รวมทั้งสิ้น			**100**

เกณฑ์การตัดสินผลการเรียน

ใช้เกณฑ์อิงเกณฑ์ตามระเบียบวัดผลของวิทยาลัย ดังนี้

คะแนนร้อยละ 80 – 100 ได้เกรด 4.0 (ดีเยี่ยม)

คะแนนร้อยละ 75 – 79 ได้เกรด 3.5 (ดีมาก)

คะแนนร้อยละ 70 – 74 ได้เกรด 3.0 (ดี)

คะแนนร้อยละ 65 – 69 ได้เกรด 2.5 (ดีพอใช้)

คะแนนร้อยละ 60 – 64 ได้เกรด 2.0 (พอใช้)

คะแนนร้อยละ 55 – 59 ได้เกรด 1.5 (อ่อน)

คะแนนร้อยละ 50 – 54 ได้เกรด 1.0 (อ่อนมาก)

คะแนนต่ำกว่าร้อยละ 50 ได้เกรด 0 (ไม่ผ่านเกณฑ์)